第 9 章 上下文、记忆与工程化工作流

> 本章目标：理解上下文窗口的预算本质与上下文污染的成因，掌握新开会话＋交接摘要、任务文件、项目指令、记忆文件与检索等治理手段，并把"一次一改、小步提交、验证后交付"变成习惯。

前几章你可能已经体会过：同一个 AI，有时一句话就改对，有时反复解释仍然改错。差别往往不在模型，而在**你放在它面前的那摞材料**。

9.1 上下文窗口：一张有限的工作台

模型并没有"记住"对话。每次你发一条消息，程序会把系统提示、历史消息、你贴的代码、工具返回的日志**重新拼成一段文本整体送进模型**，它只能看见这一段。

> 类比：上下文窗口是一张工作台，不是档案柜。档案柜（硬盘、Git、文档）可以放无限多的东西；工作台只有那么大，想用哪份材料，得先搬到台面上。台面满了，要么搬走一些，要么新的放不下。

三个直接后果：

1. **预算是共享的**。输入（对话历史＋文件内容）和输出（模型的回复）争夺同一个额度。你塞进十份文档，留给自己答案的空间就少了，长回答可能被截断。
2. **长会话更贵、更慢**。每生成一个字，模型都要"看一遍"当前全部上下文。上下文翻倍，单次生成的付出大致跟着涨，而需求并没有变得更清楚。这是 vibe coding 里最容易被忽视的开销。

3. **关键信息会"沉底"**。实践观察与公开研究都提示：信息放在上下文开头或结尾时更容易被正确使用，塞在中间长段落里的要求常常被忽略。所以"我前面不是说过了吗"经常是真的没被当回事——它被埋在中间了。

这不是缺陷清单，而是你该主动设计的约束：**让台面上只留当前有用的东西。**

9.2 上下文污染：错误如何变成"事实"

上下文里混进了不该留的东西，后续推理就会建立在错误前提上。几种典型表现：

表现	长什么样
失败尝试堆积	连着试了五种修法都报错，第六次它开始把前几种的补丁拼在一起，越改越乱
旧需求残留	需求已经从"圆形按钮"改成"方形"，它还在按圆形改样式
错误结论固化	某次它猜"这个报错是网络问题"，其实不是；之后每次都被这个猜测带着走
工具输出淹没需求	贴了几百行日志，原需求被挤到很后面，回复开始讨论无关细节

一个具体例子：第一版要"红色圆角按钮"，你改成"蓝色方形"，它改了；你再说"不要阴影，改成描边"，它又改了；第四轮你说"整体再紧凑点，用回第一版的圆角"——它很可能把红色、阴影和旧尺寸一起带回来。因为四轮全在同一窗口，"第一版"的完整描述和后几条修正平级陈列，它分不清哪条还有效。

> 要点：上下文污染不是"AI 忘了"，恰恰是它**太忠实地记着一堆互相冲突的旧指令**。治理方向不是让它记更多，而是让旧信息退出。

9.3 治理手段

各有代价，别指望一种包打天下。

手段	解决什么	代价
新开会话＋交接摘要	清除污染、重置预算	要写下状态；漏写的会被真的忘掉
任务文件 `task.md`（目标／约束／验收）	让"这一轮要什么"始终在最前面	每次需求变更要同步更新，否则文件过期
项目指令 `AGENTS.md` ／ `CLAUDE.md`	风格、命令、禁区不用反复交代	写太长挤占预算，新旧规则会冲突
`MEMORY.md`（跨会话事实）	设备参数、已确认结论、未解决风险	会过期，必须删旧不堆新
`docs/`（稳定知识）	架构、接口、部署方式	需要维护，AI 不一定主动去读
`reports/`（一次性过程）	某次任务的验证记录与结果	不该当长期知识反复塞回上下文
检索（RAG）	资料量大、按需取用	要建索引；检索不准反而引入噪声
压缩／摘要	长会话续命	有损，"不要做什么"最容易被压掉

一个实用的分工直觉：**任务文件回答"现在做什么"，项目指令回答"一直怎么做事"，记忆文件回答"已经确认了什么"，文档回答"怎么运作"，报告回答"上次做了什么"。** 五类各归其位，就不会全挤在工作台上。

9.4 交接摘要：换台面时别丢东西

新开会话是最高效的去污手段，前提是你交出一份好摘要。摘要不是聊天记录的浓缩，而是**下一轮能直接开工的状态**。可复制模板：

```
# 交接摘要
## 目标
（一句话：最终达成什么，给谁用）
## 已完成
- （已实现并验证的部分，含文件路径）
## 当前状态
- 能否运行；最后一条命令与结果；可用的环境版本
## 未解决
- （卡在哪，报错原文，已排除哪些原因）
## 下一步
- （按优先级 1-3 条，第一条要足够小）
## 不要做什么
- （不要重构、不要动某目录、不要改接口、不要加依赖）
```

最后一段最容易被省略，却最值钱。"不要做什么"给新会话划边界，缺了它，AI 很可能热情地把已验证的东西重写一遍。检验方法：**一个不了解过程的人只看这份摘要，能不能接着干？**

9.5 RAG：按需取资料

RAG（检索增强生成）解决"资料太多，塞不进上下文"的问题：

```
文档 → 切块(chunk) → 向量化(embedding) → 存入向量库
                                              ↓
提问 → 向量化 → 相似度检索 top-k 片段 → 拼进上下文 → 模型生成
```

> 类比：RAG 是图书馆员。你不必把整个图书馆搬上工作台，只要说清要查什么，他取出最相关的几页递给你。

它什么时候有用：资料量大、持续更新、答案通常落在少数几段原文里（产品手册、规章制度、笔记库、代码库问答）。

什么时候不必用：总共只有十几页，直接放进上下文更准更省事；需要通读全文做全局推理（比如"把这套接口重新设计一遍"），零散片段反而误导；资料质量差或切块切碎了语义，检索出来的比没有更糟。

在本课程的项目里，`docs/` 加关键词检索就是一种朴素 RAG：平时不读，出问题时定向查。这比"把所有文档一次性喂进去"更接近正确用法。

9.6 工程习惯：让每一步都可回退

AI 写代码快到你来不及看。约束它的不是道德劝说，而是流程：

- **一次一改**。一轮只做一件事，改完立刻验证。同时改三处，出错时分不清是谁的锅。
- **小步提交**。每完成一个可验证的小功能就提交一次；提交点就是你能回去的安全垫。
- **commit message 说清"为什么"**。`修复按钮点击无响应：事件名拼写错误` 比 `update` 有用得多。
- **diff 审阅**。提交前看改动列表：动了哪些文件、删了什么、有没有顺手改到不相关处。这是拦住"悄悄重构"的唯一时机。
- **测试或人工验证**。没有自动测试就手动跑主路径，并记下怎么跑的。
- **回滚要练过**。知道怎么撤销未提交改动、回到上一个提交、恢复被删文件。没练过的回滚等于没有。
- **分支只在必要时用**。个人小项目一路主线加勤提交通常够用；要并行不相容的改动时再开分支。
- **日志与报告**。过程写进 `reports/`，别写进 `MEMORY.md`；结论沉淀进文档或记忆。
- **权限最小化**。不给 AI"删除整个目录""直接写生产库"的便利；密钥不进对话、不进仓库。

> 要点：vibe coding 的核心不是"描述得漂亮"，而是**每走一步都留下可检查、可回退的痕迹**。改得快而不可回退，比改得慢更危险。

9.7 一个完整的小工作流

以"给网页加一个按钮"为例，看每一步的产物。

```
① 任务描述（写进 task.md）
   目标：控制面板增加"归位"按钮，点击后回到预设姿势
   约束：只改前端；复用已有动作接口；不改后端
   验收：点击后归位；按钮禁用状态正确；无控制台报错

② 方案（对话里确认，不落盘）
   "新增按钮，调用现有动作函数，执行期间禁用，完成后恢复并提示"
   → 你确认可行 → 开工

③ 改动（AI 执行，你审 diff）
   只动了控制面板页面与按钮样式，未触及后端
   你检查：有没有顺手改别的？事件绑定对不对？异常分支漏没漏？

④ 测试（人跑一遍）
   点一次 → 归位成功
   连点两次 → 第二次被忽略
   断开设备再点 → 报错提示，页面不卡死

⑤ 提交
   git commit -m "增加归位按钮：复用动作接口并处理执行期禁用"

⑥ 报告（reports/ 下一条）
   完成项 / 改动文件 / 验证方式与结果 / 未验证的边界
```

注意 ④ 里那几个"连点两次""断开设备"——它们把"看起来能用"和"确实能用"分开了。第 11 章会看到，真实项目出问题几乎都在这类边界上。

9.8 子代理与并行

子代理（subagent）是主对话派出去的独立工作单元：它**有自己的上下文窗口**，做完只把结论带回来。

适合开子代理	理由
大范围搜索	翻几十个文件找一处实现，结论一句话就够
非核心脏活	批量改名、格式转换、机械整理
独立评审	让另一个视角专门挑毛病，不受你的思路影响
几条不相干的线	同时推进，互不污染

不适合开	理由
一两行的小改动	交接成本高于收益
危险硬件或系统级操作	责任要落在你手上
需要大量来回确认的设计讨论	沟通比一个人做还慢

上下文隔离就是它的核心价值：一次失败的搜索会留下大量错误线索，全在主对话里就会污染后续；放在子代理里，失败的尝试随它一起结束。代价是它看不见主对话细节，所以派活时要把目标、约束、验收说全，并**复核它带回来的结论**——它也可能自信地报告一个没验证过的东西。

课后题

1. 打开你最近一次卡住的 AI 对话，找出至少两条"过期但仍在上下文里"的信息，说明它们怎么让后续回复跑偏的。
2. 为一次中断的任务写交接摘要：用 9.4 的模板把当前项目状态写一份，"不要做什么"至少两条；

然后新开会话只贴摘要，看它能否接着干。

3. 给一个正在做的项目建立五类文件的分工：`task.md`、项目指令、`MEMORY.md`、`docs/`、`repo rts/`，各写一句话说明放什么、不放什么。
4. 判断下列场景该不该用 RAG：（a）一份二十页的设备说明书；（b）三千份历史工单里找相似故障；（c）让 AI 把整个项目的接口重新设计一遍。
5. 你让 AI 同时改了三处功能，出了 bug 却分不清来源。复盘：重来你会把改动切成几段、每段怎么验证？
6. 开放题：有人说"上下文越长越好，反正窗口很大"。结合 9.1 和 9.2 写一段反驳，并举一个你愿意"宁可少贴"的例子。